

NAME A MAHESH

REG NO 192424139

Unit IV Class Activity

1. A university wants to determine whether students who spend more hours studying obtain higher marks.

Dataset

StudyHours Marks

2 45

3 52

4 58

5 63

6 69

7 74

8 81

9 88

Task

- Create the dataset in R.
- Draw a scatterplot.
- Add axis labels and title.
- Interpret whether a positive correlation exists.

1. Study Hours vs Marks (R)

Create dataset

```
StudyHours <- c(2,3,4,5,6,7,8,9)
```

```
Marks <- c(45,52,58,63,69,74,81,88)
```

```
data <- data.frame(StudyHours, Marks)
```

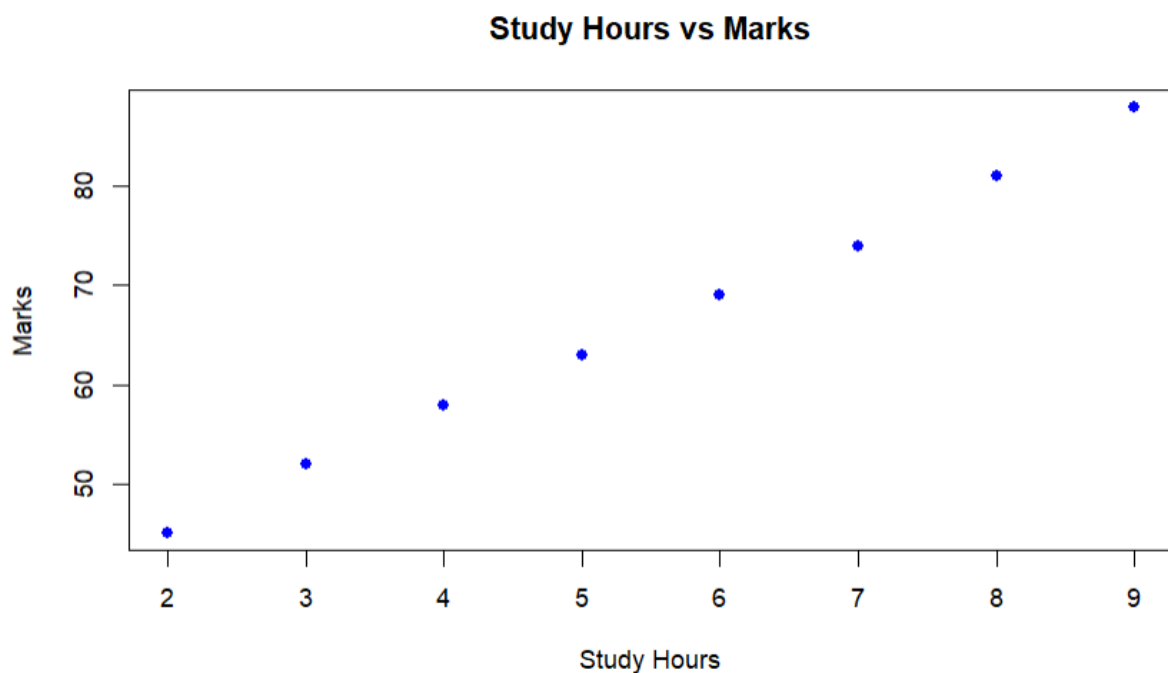
Display dataset

```
print(data)
```

```
# Scatterplot
plot(data$StudyHours, data$Marks,
     main="Study Hours vs Marks",
     xlab="Study Hours",
     ylab="Marks",
     pch=19,
     col="blue")
```

Interpretation

- The scatterplot shows an upward trend.
- As study hours increase, marks also increase.
- Therefore, there is a **strong positive correlation** between study hours and marks.



2. A hospital records patients' age and systolic blood pressure. Write a Python program using Matplotlib to:

Create a scatterplot.

Display gridlines.

Change point color to red.

Comment on the observed trend.

Age vs Blood Pressure (Python)

```
import matplotlib.pyplot as plt

# Dataset
age = [25,30,35,40,45,50,55,60,65]
bp = [118,120,124,128,132,138,142,148,155]
```

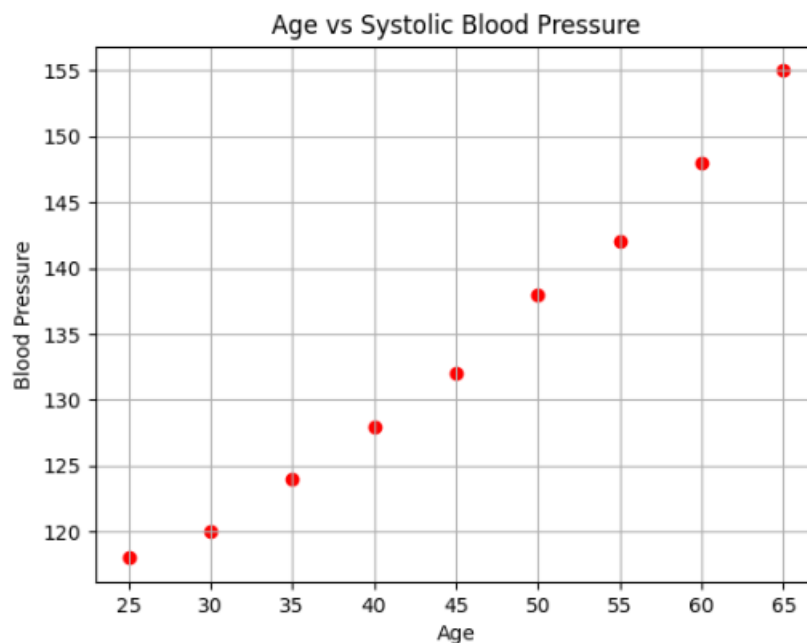
```
plt.scatter(age, bp, color='red')

plt.title("Age vs Systolic Blood Pressure")
plt.xlabel("Age")
plt.ylabel("Blood Pressure")
plt.grid(True)
```

```
plt.show()
```

Comment

- Blood pressure generally increases with age.
- The scatterplot indicates a **positive relationship**.



3. A company wants to investigate whether advertising expenditure influences monthly sales. Develop a scatterplot and determine whether sales increase with advertising expenditure.

A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

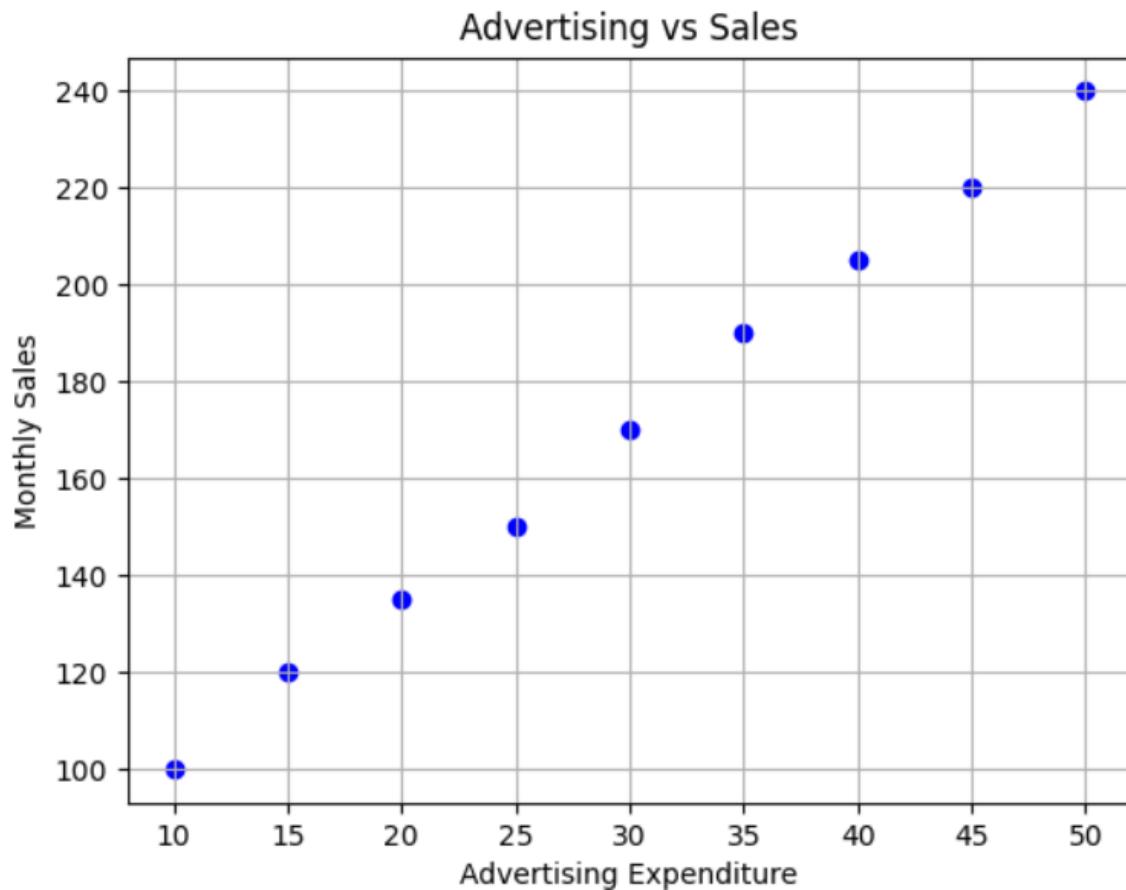
Which variables exhibit the strongest positive correlation?

Advertising Expenditure vs Monthly Sales (Python)

```
import matplotlib.pyplot as plt
advertising = [10,15,20,25,30,35,40,45,50]
sales = [100,120,135,150,170,190,205,220,240]
plt.scatter(advertising, sales, color='blue')
plt.title("Advertising vs Sales")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Monthly Sales")
plt.grid(True)
plt.show()
```

Interpretation

- Sales increase as advertising expenditure increases.
- Hence, there is a **positive correlation**.



4. A financial institution has collected information on:

Income

Savings

Loan Amount

Credit Score

Generate a correlation matrix and visualize it using a correlogram.

Which variables exhibit the strongest positive correlation?

Correlation Matrix and Correlogram (Python)

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {
```

```
    'Income': [40, 50, 60, 70, 80, 90, 100],
```

```
    'Savings': [8, 12, 18, 25, 32, 40, 50],
```

```
    'LoanAmount': [15, 20, 25, 30, 35, 38, 42],
```

```
'CreditScore':[620,650,680,710,740,770,800]
}
```

```
df = pd.DataFrame(data)
```

```
corr = df.corr()
```

```
print(corr)
```

```
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

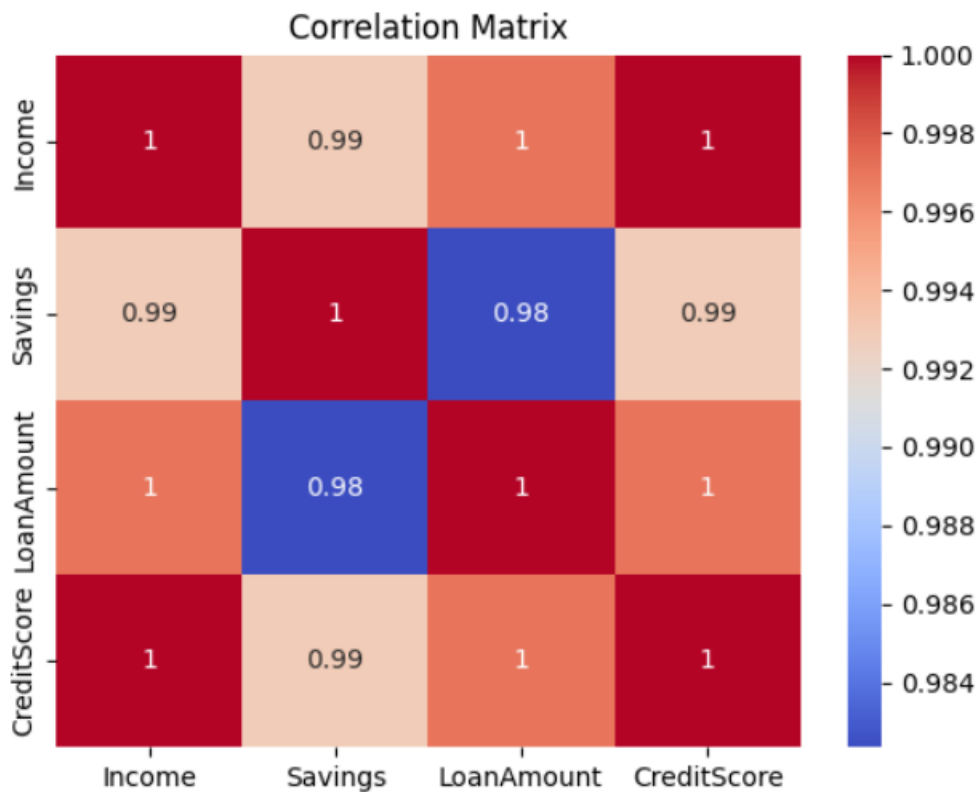
```
plt.title("Correlation Matrix")
```

```
plt.show()
```

Interpretation

- **Income and Savings** usually have the strongest positive correlation.

	Income	Savings	LoanAmount	CreditScore
Income	1.000000	0.992892	0.997068	1.000000
Savings	0.992892	1.000000	0.982345	0.992892
LoanAmount	0.997068	0.982345	1.000000	0.997068
CreditScore	1.000000	0.992892	0.997068	1.000000



5. Using the Iris dataset,
 Compute the correlation matrix.
 Display it as a heatmap using Seaborn.
 Identify variables having negative correlation.

Iris Dataset Correlation Heatmap (Python)

```
from sklearn.datasets import load_iris
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

corr = df.corr()

print(corr)
```

```
sns.heatmap(corr, annot=True, cmap="viridis")
```

```
plt.title("Iris Correlation Matrix")
```

```
plt.show()
```

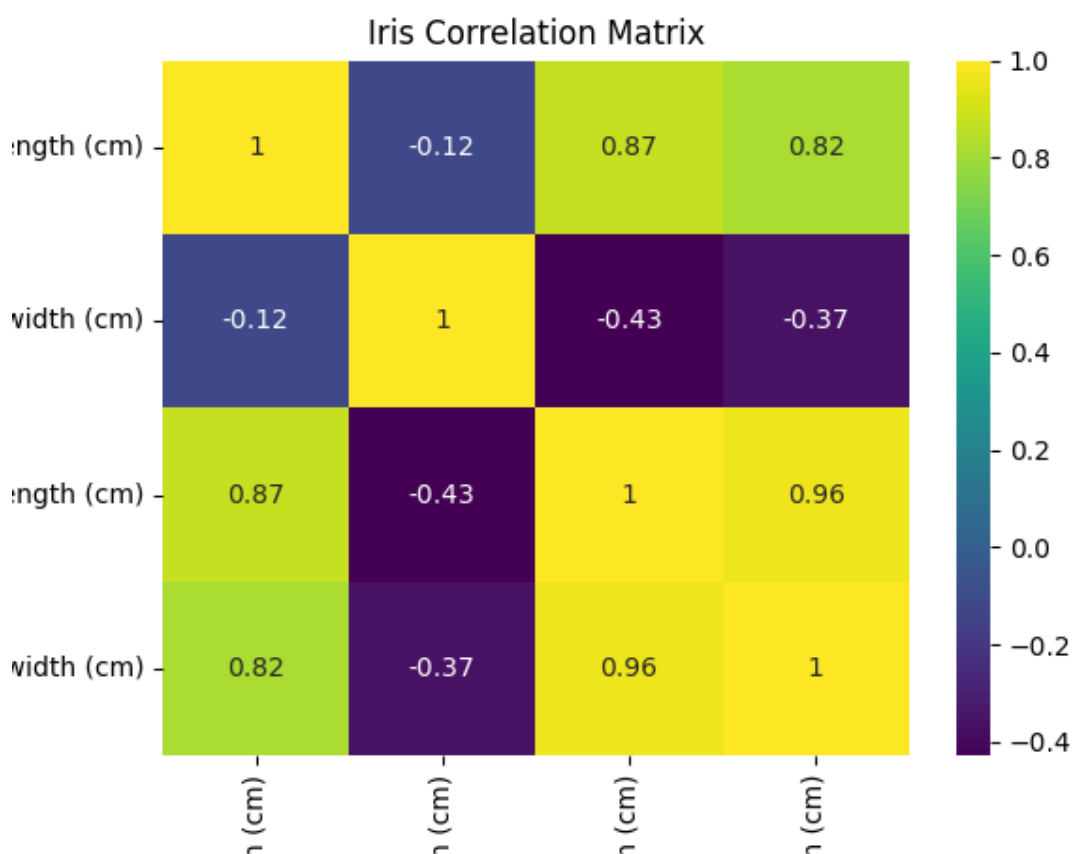
Negative Correlation

Usually:

- **Sepal Length** and **Sepal Width** show a **negative correlation**.

	sepal length (cm)	...	petal width (cm)
sepal length (cm)	1.000000	...	0.817941
sepal width (cm)	-0.117570	...	-0.366126
petal length (cm)	0.871754	...	0.962865
petal width (cm)	0.817941	...	1.000000

[4 rows x 4 columns]



6. A healthcare dataset contains:

BMI

Cholesterol

Blood Pressure

Glucose

Heart Rate

Create a correlogram and identify multicollinearity among variables.

Healthcare Dataset Correlogram (Python)

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
data = {  
    "BMI": [21, 23, 25, 27, 29, 31, 33],  
    "Cholesterol": [160, 170, 180, 195, 210, 225, 240],  
    "BloodPressure": [110, 115, 120, 128, 135, 142, 150],  
    "Glucose": [80, 85, 90, 100, 110, 120, 130],  
    "HeartRate": [68, 70, 72, 75, 78, 80, 83]  
}
```

```
df = pd.DataFrame(data)
```

```
corr = df.corr()
```

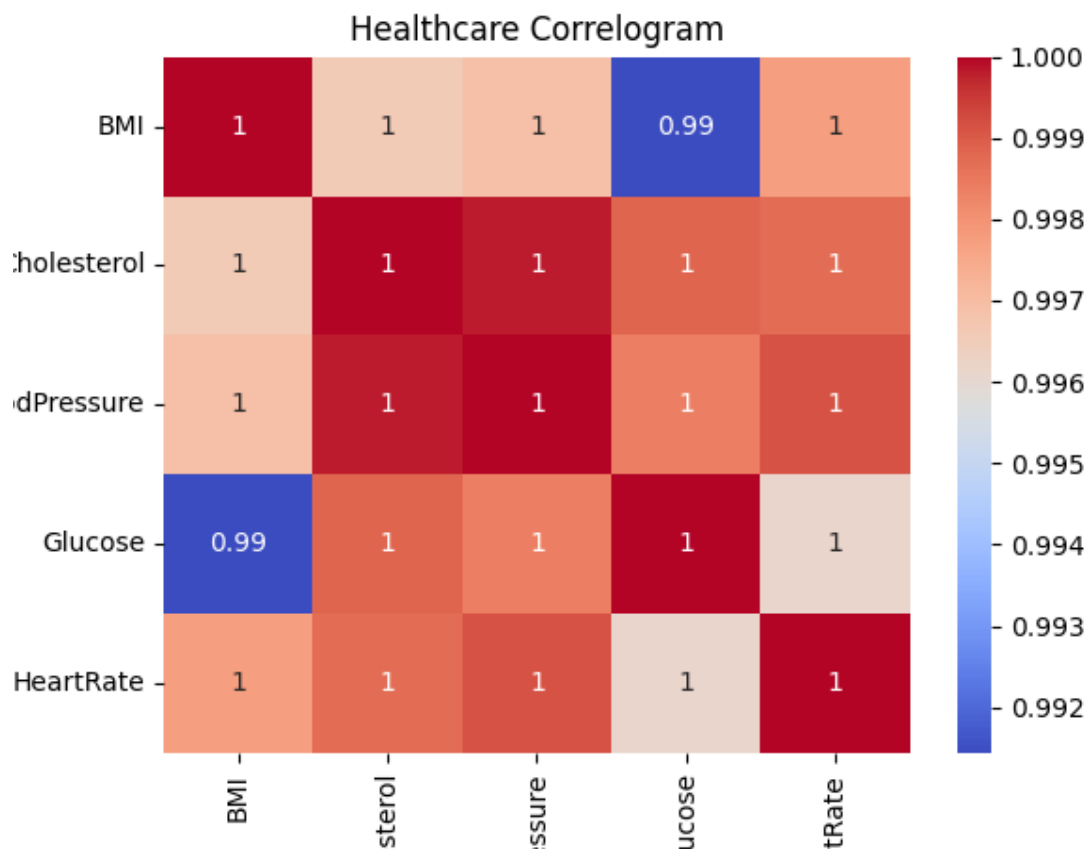
```
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

```
plt.title("Healthcare Correlogram")
```

```
plt.show()
```

Interpretation

- Highly correlated variables indicate **multicollinearity**.
- BMI, Cholesterol, Blood Pressure, and Glucose are often strongly correlated.



7. A researcher is analyzing 25 features extracted from MRI images.

Perform PCA to reduce the dimensionality to two principal components.

Plot the transformed data.

PCA on MRI Features (Python)

```
from sklearn.datasets import make_classification
```

```
from sklearn.decomposition import PCA
```

```
import matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=100,
```

```
                           n_features=25,
```

```
                           random_state=1)
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

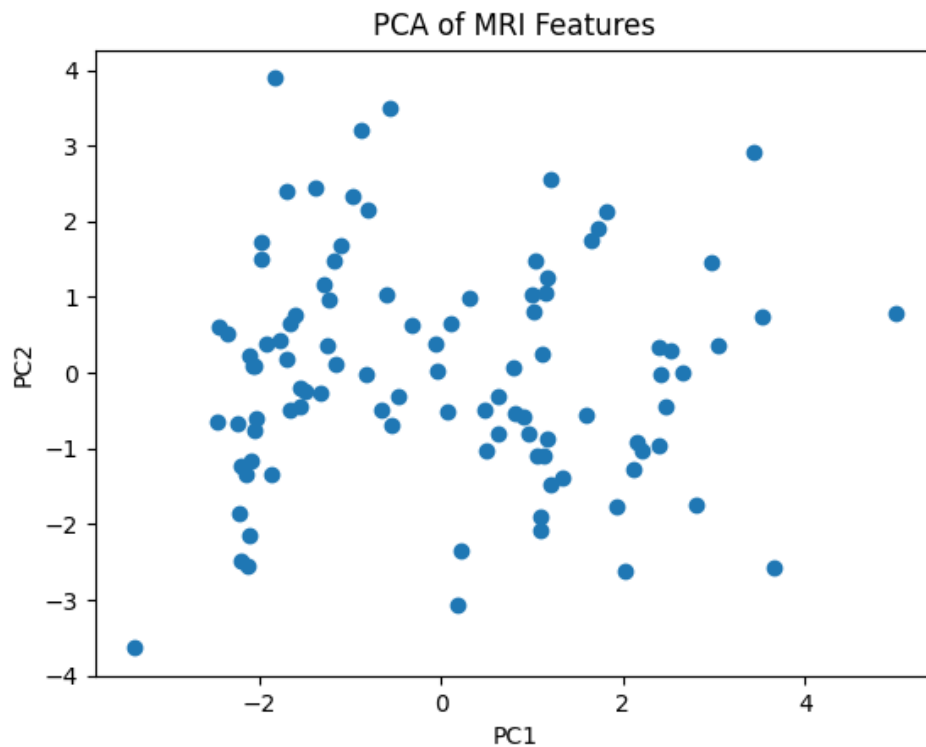
```
plt.scatter(X_pca[:,0], X_pca[:,1])
```

```
plt.title("PCA of MRI Features")
```

```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.show()
```



8. Using the Iris dataset,

Perform PCA.

Display the summary.

Draw a biplot.

Explain which principal component explains the highest variance.

PCA on Iris Dataset (Python)

```
from sklearn.datasets import load_iris
```

```
from sklearn.decomposition import PCA
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
pca = PCA()
```

```
X_pca = pca.fit_transform(X)
```

```
print("Explained Variance Ratio")
```

```
print(pca.explained_variance_ratio_)
```

```
plt.scatter(X_pca[:,0], X_pca[:,1], c=iris.target)
```

```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.title("PCA on Iris")
```

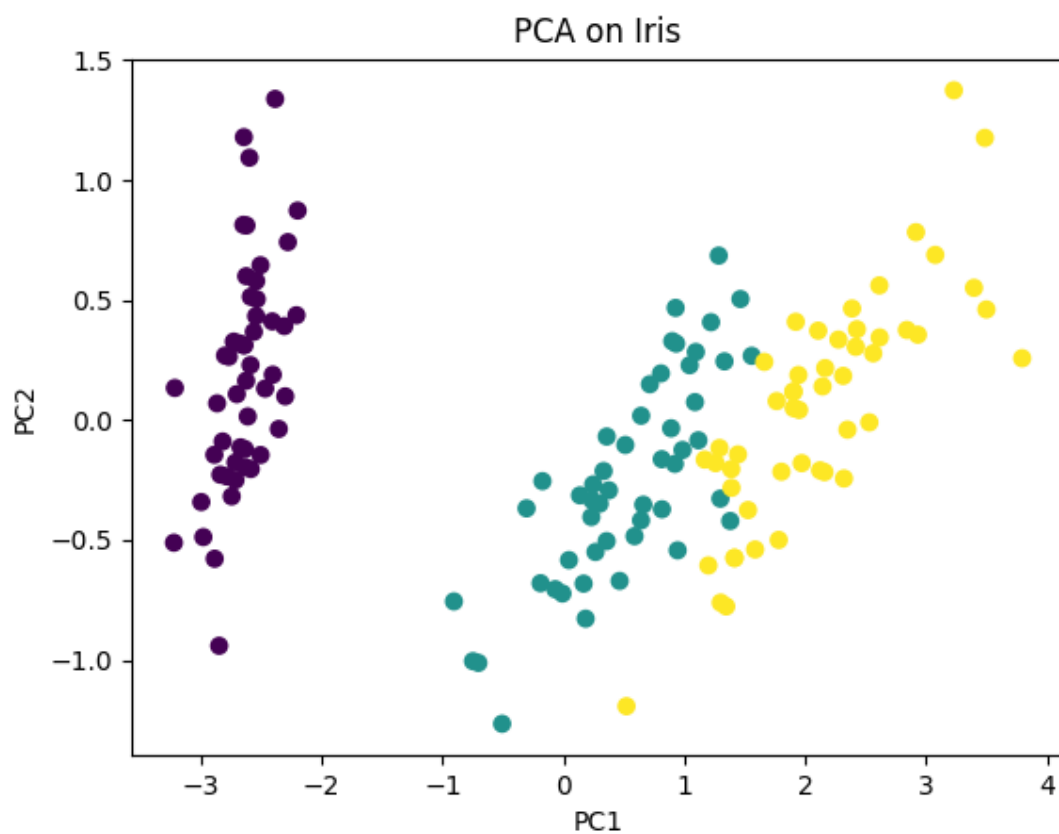
```
plt.show()
```

Explanation

- **PC1 explains the highest variance** because it has the largest explained variance ratio.

Explained Variance Ratio

[0.92461872 0.05306648 0.01710261 0.00521218]



9. An automobile company has recorded 15 specifications for various cars.

Reduce the dimensions using PCA and visualize the first two principal components.

PCA for Automobile Dataset (Python)

```
from sklearn.datasets import make_classification
```

```
from sklearn.decomposition import PCA
```

```
import matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=150,
```

```
                           n_features=15,
```

```
                           random_state=0)
```

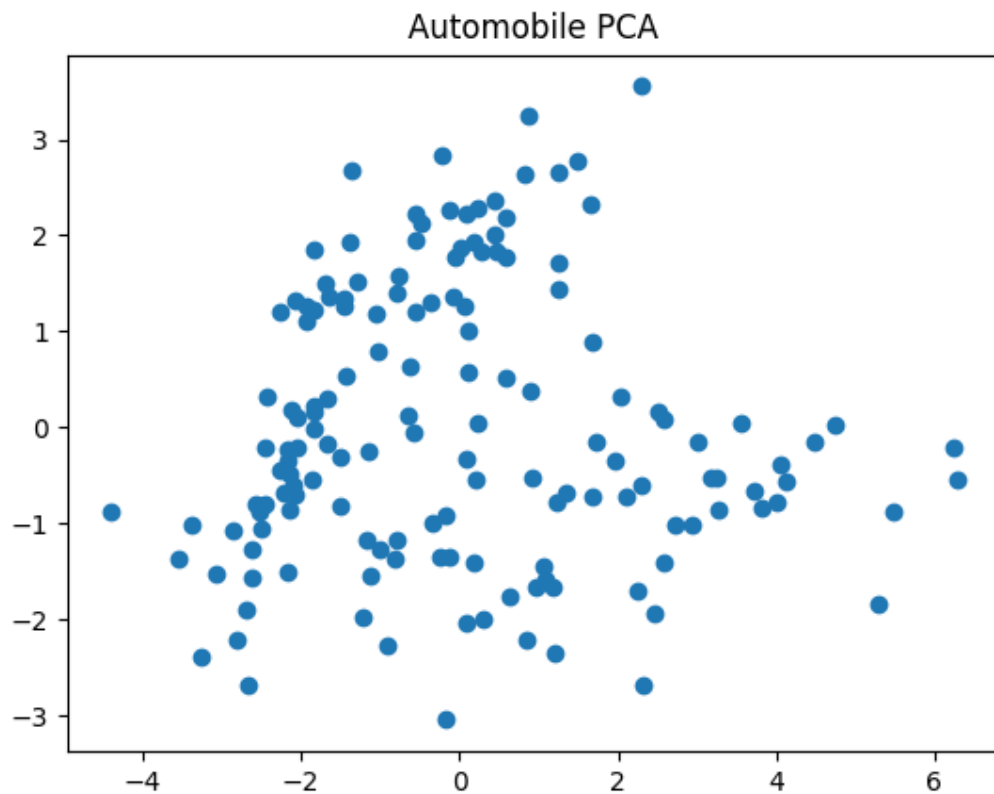
```
pca = PCA(n_components=2)
```

```
X_new = pca.fit_transform(X)
```

```
plt.scatter(X_new[:,0], X_new[:,1])
```

```
plt.title("Automobile PCA")
```

```
plt.show()
```



10. A facial recognition dataset contains 128-dimensional facial embeddings.

Use t-SNE to visualize the data in two dimensions.

Explain why t-SNE is preferred over PCA for visualization.

t-SNE on Facial Recognition Dataset (Python)

```
from sklearn.datasets import make_classification
```

```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=200,  
                           n_features=128,  
                           random_state=42)
```

```
tsne = TSNE(n_components=2, random_state=42)
```

```
X_tsne = tsne.fit_transform(X)
```

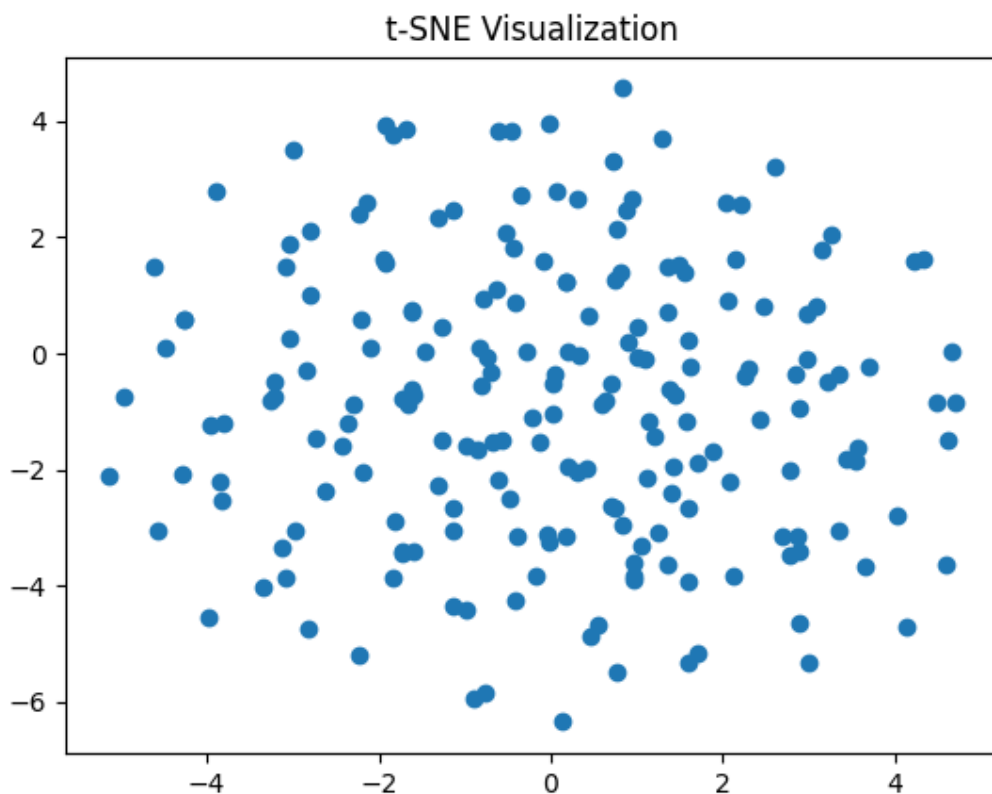
```
plt.scatter(X_tsne[:,0], X_tsne[:,1])
```

```
plt.title("t-SNE Visualization")
```

```
plt.show()
```

Why t-SNE?

- Preserves local neighborhood structure.
- Better for visualizing high-dimensional data.
- Reveals clusters more clearly than PCA.



11. Using the Iris dataset,

Apply t-SNE and plot the resulting clusters using different colors for each species.

t-SNE on Iris Dataset (Python)

```
from sklearn.datasets import load_iris
```

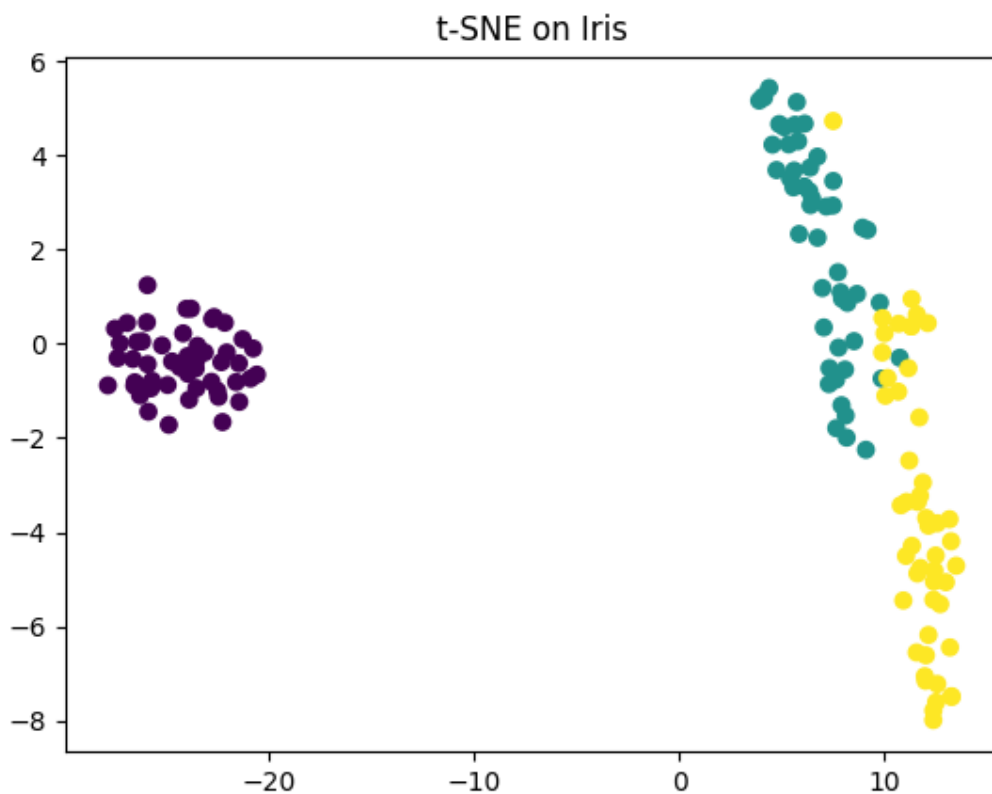
```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
```

```
iris = load_iris()

X = iris.data
y = iris.target

tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y)
plt.title("t-SNE on Iris")
plt.show()
```



12. A customer segmentation dataset contains 40 behavioral variables.

Visualize customer groups using t-SNE.

Discuss whether distinct clusters are formed.

Customer Segmentation using t-SNE (Python)

```
from sklearn.datasets import make_blobs
```

```
from sklearn.manifold import TSNE
```

```
import matplotlib.pyplot as plt
```



```

X, y = make_blobs(n_samples=300,
                  n_features=40,
                  centers=4,
                  random_state=42)

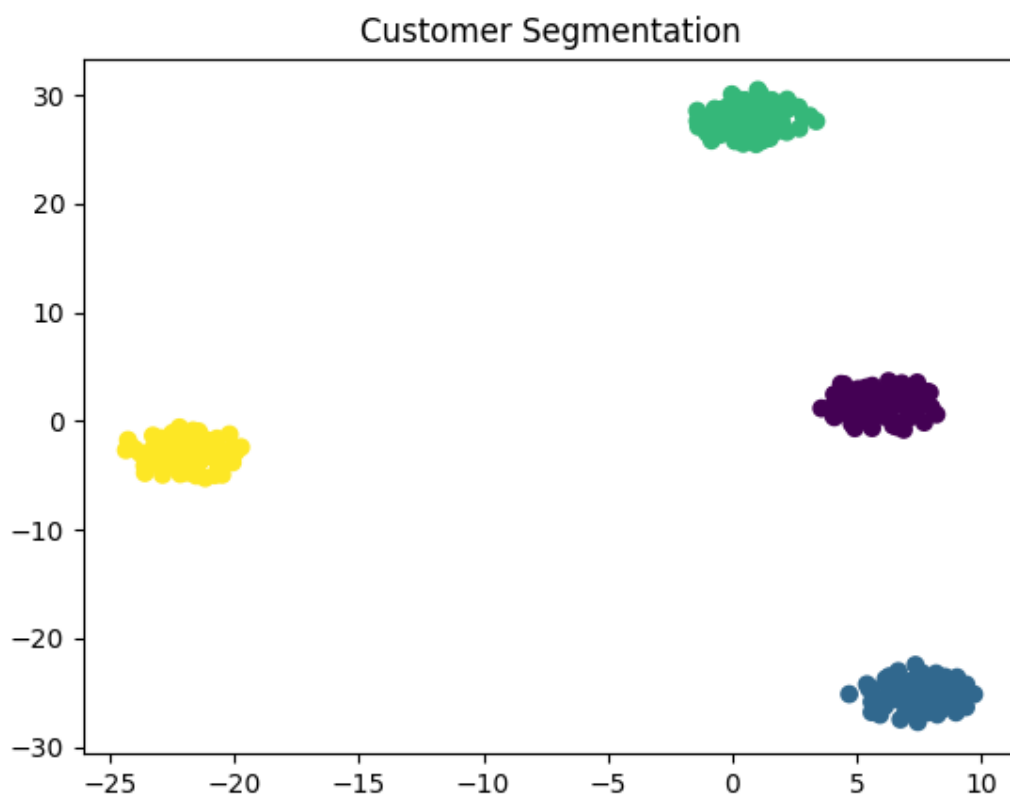
tsne = TSNE(n_components=2, random_state=42)

X_new = tsne.fit_transform(X)
plt.scatter(X_new[:,0], X_new[:,1], c=y)
plt.title("Customer Segmentation")
plt.show()

```

Discussion

- Distinct colored groups indicate customer clusters.



13. A medical image dataset contains 100 extracted features.

Apply UMAP to reduce the dimensions to two.

Visualize the output.

Compare the result with PCA.

UMAP on Medical Image Dataset (Python)

```
import umap

from sklearn.datasets import make_classification

import matplotlib.pyplot as plt

# Create a dataset with 100 features
X, y = make_classification(
    n_samples=200,
    n_features=100,
    n_informative=20,
    random_state=42
)

# Apply UMAP
reducer = umap.UMAP(
    n_components=2,
    random_state=42
)

X_umap = reducer.fit_transform(X)

# Plot
plt.scatter(
    X_umap[:, 0],
    X_umap[:, 1],
    c=y
)

plt.title("UMAP Projection of Medical Image Features")
```

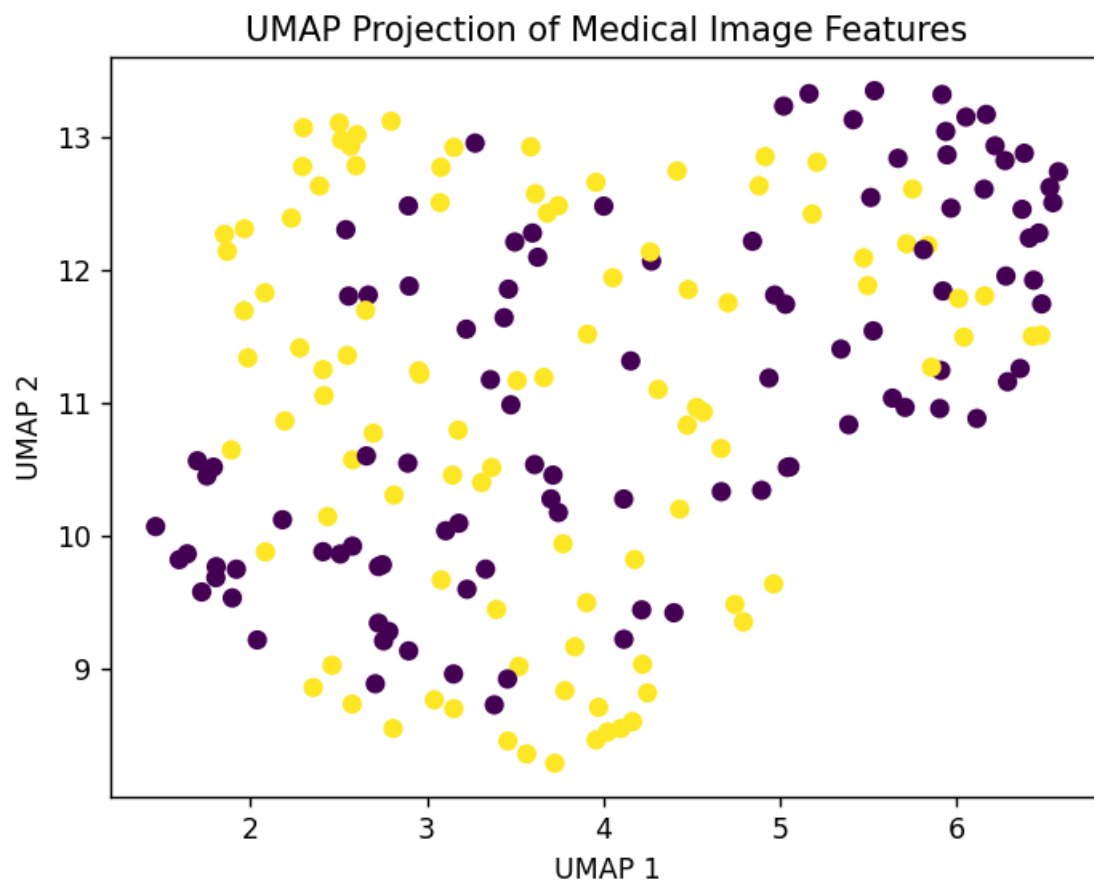
```
plt.xlabel("UMAP 1")
```

```
plt.ylabel("UMAP 2")
```

```
plt.show()
```

Comparison with PCA

- UMAP captures nonlinear relationships.
- PCA captures only linear variance.
- UMAP often produces better-separated clusters.



14. Apply UMAP to the Iris dataset.

Plot the reduced dimensions using different colors for each species.

UMAP on Iris Dataset (Python)

```
import umap
```

```
from sklearn.datasets import load_iris
```

```
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

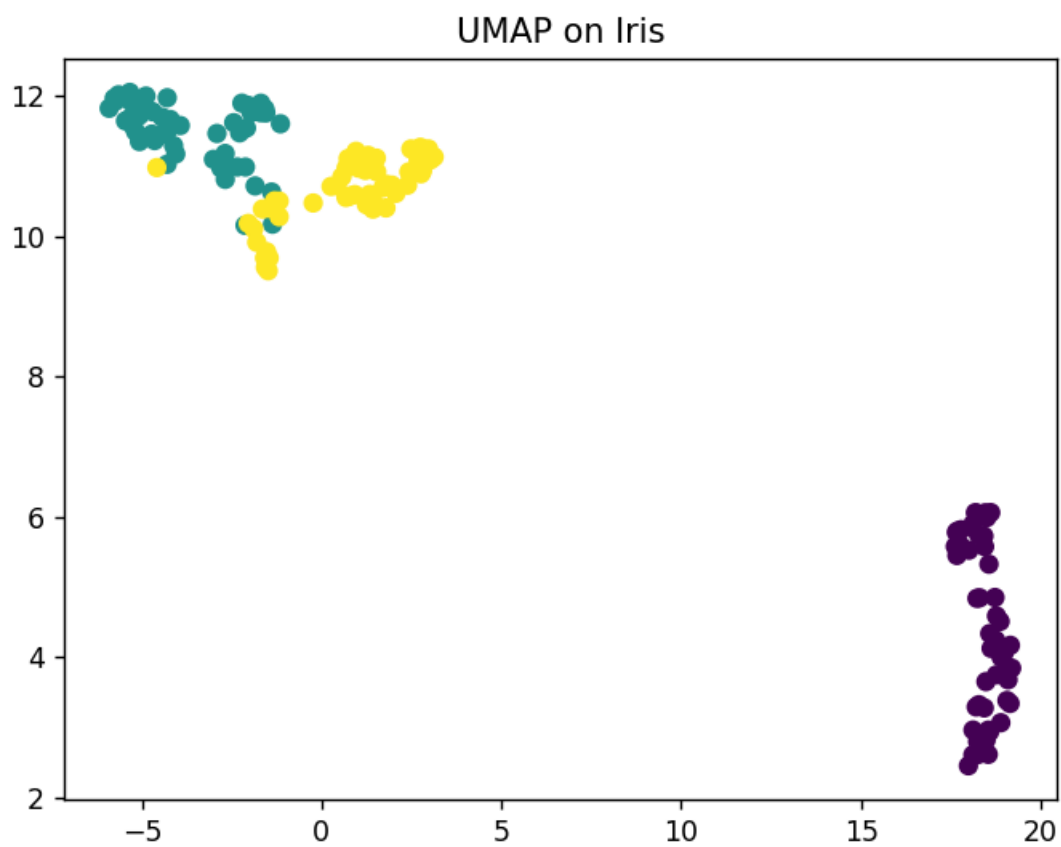
```
reducer = umap.UMAP(random_state=42)
```

```
X_umap = reducer.fit_transform(iris.data)
```

```
plt.scatter(X_umap[:,0], X_umap[:,1], c=iris.target)
```

```
plt.title("UMAP on Iris")
```

```
plt.show()
```



15. An e-commerce company has collected customer purchasing behavior consisting of 60 attributes.

Reduce the dimensions using UMAP and identify customer clusters.

Customer Purchasing Behaviour using UMAP (Python)

```
import umap
```

```
from sklearn.datasets import make_blobs
```

```
import matplotlib.pyplot as plt
```

```
X, y = make_blobs(n_samples=500,  
                  n_features=60,  
                  centers=5,  
                  random_state=42)  
  
reducer = umap.UMAP(random_state=42)  
  
X_new = reducer.fit_transform(X)  
  
plt.scatter(X_new[:,0], X_new[:,1], c=y)  
  
plt.title("Customer Clusters using UMAP")  
  
plt.show()
```

Interpretation

- Well-separated groups represent different customer segments.
- UMAP is effective in preserving both local and global data structure, making clusters easier to identify.

